

CO2060 · Group e22 · Debate Management System (VIVAATHI)

Test Catalogue

Every automated test in the project: how it was conducted, what it checks, and its result

Companion to the Testing Report · run of 25 September 2026 · branch Testing

Contents

1. How to read this catalogue	4
2. How each test suite was conducted.....	4
3. Backend unit tests (JUnit 5 + Mockito)	5
JwtUtilTest: JwtUtil: login token creation and validation	5
AuthServiceTest: AuthService: sign-up and login rules	5
ScoreSheetServiceTest: ScoreSheetService: judge scoring and match results	6
TournamentAndMatchServiceTest › TournamentTests: TournamentService: creating tournaments and judges	7
TournamentAndMatchServiceTest › MatchTests: MatchService: creating matches and knockout rounds.....	7
SocialServicesTest › Connections: ConnectionService: connect, accept, block	8
SocialServicesTest › Diaries: DiaryService: likes, verification, deletion	8
SocialServicesTest › Stats: StatsService: leaderboard and statistics	9
4. Backend integration tests (Spring Boot + MockMvc + H2)	10
AuthApiIntegrationTest: Authentication API.....	10
RbacIntegrationTest: Role-based access control (RBAC).....	10
TournamentWorkflowIntegrationTest: Full workflows	14
OwnershipSecurityIntegrationTest: Ownership and data exposure.....	15
5. Frontend component tests (Vitest + React Testing Library)	16
LoginPage.test.tsx: Login page	16
SignupPage.test.tsx: Sign-up and role selection pages	16
App.test.tsx: Protected routes (frontend access rules)	17
AuthContext.test.tsx: AuthContext (session state)	18
axios.test.ts: API client (axios interceptors)	19
utils.test.tsx: Utilities and toast	19
6. API end-to-end tests (Postman collection run by Newman)	21
01 Health & public pages	21
02 Authentication	21
03 Role-based access control	22

04 Tournament lifecycle	23
05 Social features.....	25
06 Ownership & privacy	27
7. Browser end-to-end tests (Playwright on Microsoft Edge)	28
01-auth.spec.ts: Authentication and access (browser)	28
02-session.spec.ts: Multi-tab sessions (browser)	28
03-tournament.spec.ts: Tournament workflow (browser, run in order).....	29
04-public.spec.ts: Public pages (browser).....	29
05-score-sheet.spec.ts: Score sheet (browser, independent tests)	29
8. Lighthouse audits.....	30

1. How to read this catalogue

- **ID prefixes:** BU = backend unit, BI = backend integration, FE = frontend, API = Postman request, E2E = browser test, LH = Lighthouse audit.
- **Test:** a plain-language name; the method name in the source code is shown in brackets for backend tests.
- **How it was conducted:** the data prepared and the action performed.
- **What is checked:** the assertions. For a failing test, this column also gives the expected and actual result and the defect number (DEF-xx) from the Testing Report.
- **Result:** taken automatically from the result files of the latest run, not typed by hand. Every failure is a confirmed application defect; no test was weakened to make it pass.

2. How each test suite was conducted

Suite	Environment	Test data	Command	Result file
Backend unit	JDK 25, Maven; Mockito fakes, no database	Built in memory in each test	cd backend && mvn test	backend/target/surefire-reports, jacoco
Backend integration	Full Spring Boot app; in-memory H2 in PostgreSQL mode; MockMvc	Users created through the real sign-up endpoint	cd backend && mvn test	Same as above
Frontend	Node 24, Vitest, jsdom, React Testing Library	Mocked API responses; fresh localStorage per test	cd frontend && npx vitest run --coverage	frontend/test-results
API end-to-end	Backend jar on :8081 + PostgreSQL 18 database dms_test; Newman	Created by the collection with a unique run id	cd testing && npm run api	testing/reports/newman
Browser end-to-end	Backend :8081 + Vite frontend :5174 + dms_test; Playwright with Microsoft Edge	Four fresh users created by global setup	cd testing && npx playwright test	testing/reports/playwright
Lighthouse	Production build via vite preview; headless Edge	Seeded demo data	npx lighthouse <url> --preset=desktop	testing/reports/lighthouse

Totals in this catalogue: 262 tests and requests, 228 passed, 34 failed. Lighthouse audits are listed separately.

Suite	Tests	Passed	Failed
Backend unit (BU)	46	46	0
Backend integration (BI)	70	55	15
Frontend (FE)	52	52	0
API end-to-end requests (API)	69	54	15
Browser end-to-end (E2E)	25	21	4

3. Backend unit tests (JUnit 5 + Mockito)

Each service class is created on its own and every repository it depends on is replaced by a Mockito fake. The test decides what each fake returns, calls one service method, and then checks the returned value, the thrown error, and which fakes were called. No Spring context and no database are used, so all 46 tests run in about 2 seconds.

JwtUtilTest: JwtUtil: login token creation and validation

How this group was set up: A JwtUtil object is created directly with a test secret and a 60-second lifetime. No Spring, no database.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-01	Generated token is valid and carries the username (generateAndValidate)	Create a token for "debater1" with role DEBATER, then validate it and read the username back.	Token is not blank and has 3 dot-separated parts; validateToken returns true; extractUsername returns "debater1".	Pass
BU-02	Expired token is rejected (expiredTokensIsRejected)	A second JwtUtil with a lifetime of -1 second creates a token that is already expired; the normal JwtUtil validates it.	validateToken returns false.	Pass
BU-03	Tampered token is rejected (tamperedTokensIsRejected)	Take a debater's token and swap its middle part (payload) for an organizer's payload, keeping the original signature — simulating someone editing their role.	validateToken returns false (signature no longer matches).	Pass
BU-04	Token signed with a different secret is rejected (tokenFromOtherSecretIsRejected)	An "attacker" JwtUtil with a different secret creates an ORGANIZER token; our JwtUtil validates it.	validateToken returns false.	Pass
BU-05	Garbage, empty and null tokens are rejected (garbageTokensAreRejected)	Validate the strings "not-a-jwt", "" and null.	All three return false and no exception is thrown.	Pass

AuthServiceTest: AuthService: sign-up and login rules

How this group was set up: AuthService runs with Mockito fakes for UserRepository, DebaterStatsRepository, JudgeStatsRepository, PasswordEncoder and JwtUtil, so only the service logic is exercised.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-06	Debater sign-up hashes password, creates stats, returns token (signupDebater)	Call signup() for a DEBATER; the fake encoder returns "HASH" and the fake JwtUtil returns "TOKEN".	Response has token "TOKEN", username and role DEBATER; the saved user has password hash "HASH" (never the plain password); a DebaterStats row is saved; judge stats are never touched.	Pass
BU-07	Judge sign-up creates judge stats (signupJudge)	Call signup() for a JUDGE.	A JudgeStats row is saved; debater stats are never touched.	Pass
BU-08	Organizer sign-up creates no stats rows (signupOrganizer)	Call signup() for an ORGANIZER.	Neither stats repository is used.	Pass
BU-09	Taken username is rejected (signupDuplicateUsername)	The fake repository says the username already exists; call signup().	Error "Username already taken"; nothing is saved.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-10	Registered e-mail is rejected (signupDuplicateEmail)	The fake repository says the e-mail already exists; call signup().	Error "Email already registered"; nothing is saved.	Pass
BU-11	Login with username and correct password (loginWithUsername)	Fake repository finds user "deb"; fake encoder says the password matches.	A token is returned.	Pass
BU-12	Login with e-mail falls back to e-mail lookup (loginWithEmail)	Lookup by username finds nothing, lookup by e-mail finds the user.	Login succeeds and returns user "deb".	Pass
BU-13	Wrong password is rejected (loginWrongPassword)	User exists but the fake encoder says the password does not match.	Error "Invalid credentials"; no token is generated.	Pass
BU-14	Unknown user gets the same generic error (loginUnknownUser)	Neither username nor e-mail lookup finds a user.	Error is the same "Invalid credentials" text, so attackers cannot tell which usernames exist.	Pass
BU-15	getMe returns the current user (getMe)	Fake repository returns user with id 7.	Returned profile has id 7.	Pass

ScoreSheetServiceTest: ScoreSheetService: judge scoring and match results

How this group was set up: In-memory objects are built for an organizer, two judges (judge1, judge2), two debaters, two schools (Royal = proposition, Trinity = opposition) and one match with both judges assigned. All ten repositories are Mockito fakes.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-16	A judge cannot submit on behalf of another judge (rejectsImpersonation)	Logged in as judge1, submit a score sheet whose judgeId is judge2.	Error "only submit scores as yourself"; nothing is saved.	Pass
BU-17	A judge not assigned to the match cannot submit (rejectsUnassignedJudge)	The fake repository says judge1 is not assigned to the match; judge1 submits.	Error "not assigned".	Pass
BU-18	A judge cannot submit twice (rejectsDoubleSubmission)	The fake repository says judge1 already submitted; judge1 submits again.	Error "already submitted".	Pass
BU-19	First of two judges submitting leaves the match open (partialSubmissionKeepsMatchOpen)	Only judge1 submits (70 vs 60, best speaker debater A); judge2 has not.	judge1 is marked submitted; match stays UPCOMING with no winner; the organizer is notified "Score Sheet Submitted".	Pass
BU-20	When all judges submit, the winner is decided by average score (allSubmittedCompletesMatch)	judge1 already submitted (70 vs 60). judge2 now submits 50 vs 75. Both chose debater B as best speaker. Averages: proposition 60, opposition 67.5.	Match becomes COMPLETED; winner is Trinity (opposition); best speaker is debater B; B's stats get 1 win, 1 match, 1 best-speaker award; A's stats get 1 loss; organizer is notified "Match Completed ... Trinity".	Pass
BU-21	A tie goes to the proposition (tieGoesToProposition)	Two submissions: 60 vs 70 and 70 vs 60 (averages equal); no best speaker chosen.	Winner is the proposition school (the rule in the code: prop ≥ opp); best speaker is empty.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-22	Reopening a sheet un-marks the judge (reopen)	A submitted sheet for judge1 is reopened by id.	Sheet is flagged reopened; judge1 is no longer marked as submitted, so they can score again.	Pass
BU-23	Reopening a missing sheet reports "not found" (reopenMissing)	Reopen id 99, which the fake repository does not have.	Error contains "not found" (the API turns this into HTTP 404).	Pass

TournamentAndMatchServiceTest › TournamentTests: TournamentService: creating tournaments and judges

How this group was set up: Fake repositories return an organizer ("org"), a judge (id 10), a debater (id 20) and tournament 100. Saving returns the object with an id.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-24	Creating a tournament saves everything and notifies everyone (createFull)	createTournament() with name "Nationals", one school "Royal" containing the debater, one judge and a score template.	Tournament saved with name Nationals and the organizer; debater linked to the school; judge saved with code JUDGE-001; template saved; notifications sent to the debater ("Tournament Assigned"), judge ("Tournament Judge Assignment") and organizer ("Tournament Created").	Pass
BU-25	Unknown debater id is rejected (unknownDebater)	Create a tournament with debater id 999, which does not exist.	Error "Debater not found".	Pass
BU-26	The same debater cannot be in two schools of one tournament (duplicateDebater)	The fake repository says the debater is already assigned in this tournament.	Error "already assigned".	Pass
BU-27	Adding a judge gets the next judge code (addJudgeCode)	The tournament already has 2 judges; add another.	New judge code is JUDGE-003.	Pass
BU-28	Adding the same judge twice is rejected (addJudgeTwice)	The fake repository says this judge is already in the tournament.	Error "already added".	Pass
BU-29	Looking up a missing tournament (missingTournament)	getTournamentById(404).	Error "Tournament not found".	Pass

TournamentAndMatchServiceTest › MatchTests: MatchService: creating matches and knockout rounds

How this group was set up: Same fakes as above plus two schools, Royal (200) and Trinity (201), in tournament 100.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-30	A school cannot debate itself (sameSchool)	createMatch() with Royal as both proposition and opposition.	Error "cannot be the same".	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-31	Creating a match sets round, code and notifies the judge (createMatch)	The tournament already has round 2 and 2 matches; create a new match with judge 10.	Match saved as round 3 with code MATCH-100-3; judge link is "/score-sheet/55/10"; judge notified "New Judging Assignment".	Pass
BU-32	Next round is blocked while matches are unfinished (nextRoundBlockedUntilComplete)	Round 1 has one match that is not completed; call generateNextRound().	Error "Not all matches".	Pass
BU-33	Next round pairs the winners (nextRoundPairsWinners)	Round 1 has two completed matches; winners are Royal and school B.	A round-2 match is saved with Royal vs B.	Pass

SocialServicesTest › Connections: ConnectionService: connect, accept, block

How this group was set up: Users alice (debater), bob (debater) and org (organizer) exist in fake repositories.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-34	Sending a request creates a PENDING connection (sendRequest)	alice sends a request to bob.	Status is PENDING; bob is notified "New Connection Request".	Pass
BU-35	Cannot connect with yourself (selfConnect)	alice sends a request to alice.	Error "Cannot connect with yourself".	Pass
BU-36	Blocked users cannot send requests (blocked)	The fake block repository says alice and bob have a block.	Error "Action blocked".	Pass
BU-37	Duplicate request is rejected (duplicate)	A connection between alice and bob already exists.	Error "already exists".	Pass
BU-38	Only the receiver can accept (onlyReceiverAccepts)	alice → bob request exists. First alice tries to accept it, then bob accepts.	alice gets "Not authorized"; after bob accepts, status is ACCEPTED.	Pass
BU-39	Blocking removes the existing connection (blockRemovesConnection)	alice and bob are connected; alice blocks bob.	The connection is deleted and a Block is saved.	Pass
BU-40	Connection status is reported correctly in every state (statuses)	Check status for: self; no connection; pending sent by alice; same from bob's side; accepted; blocked.	Returns SELF, NONE, PENDING_SENT, PENDING_RECEIVED, ACCEPTED and BLOCKED respectively.	Pass

SocialServicesTest › Diaries: DiaryService: likes, verification, deletion

How this group was set up: A diary post (id 5) written by alice exists in the fake repository.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-41	Like toggles on and off (toggleLike)	bob likes the post, then likes it again.	After the first call bob is in the likes; after the second he is removed.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-42	Only organizers can verify a post (verifyRequiresOrganizer)	bob (debater) tries to verify; then org verifies.	bob gets "Only organizers"; after org, the post is verified.	Pass
BU-43	Only the author or an organizer can delete (deletePermissions)	bob tries to delete alice's post; then alice deletes; then org deletes.	bob gets "Unauthorized"; alice and org both succeed (delete called twice).	Pass
BU-44	Sharing increments the share count (share)	An anonymous visitor shares the post.	shareCount becomes 1.	Pass

SocialServicesTest › Stats: StatsService: leaderboard and statistics

How this group was set up: Tournament with schools A, B, C. Match 1: A vs B, B wins. Match 2: B vs C, B wins. Match 3: A vs C, not finished.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BU-45	Leaderboard gives 2 points per win and sorts by points (leaderboard)	Build the tournament leaderboard.	B is first with 4 points and 100% win rate; A has 1 played, 1 loss; the unfinished match is ignored.	Pass
BU-46	Debater with no stats row gets zeros (debaterStatsDefault)	The fake repository has no stats for alice.	Returned stats show 0 matches and 0 wins instead of an error.	Pass

4. Backend integration tests (Spring Boot + MockMvc + H2)

These tests start the complete backend application: the real controllers, the real Spring Security filter chain, the JWT filter, the global exception handler and the JPA repositories. They run against an in-memory H2 database in PostgreSQL mode instead of the real database. MockMvc sends real HTTP requests into the application without opening a network port. Users are created through the real sign-up endpoint, and the tokens it returns are sent in the Authorization header, exactly as the frontend does.

AuthApiIntegrationTest: Authentication API

How this group was set up: The whole Spring Boot application starts with an empty in-memory H2 database. Requests go through MockMvc, including the real security filter chain and exception handler.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-01	Sign-up returns a token and a safe profile (signupReturnsToken)	POST /api/auth/signup for a new DEBATER "newdeb".	200 OK; token is present; username "newdeb"; role DEBATER; the response has no passwordHash field.	Pass
BI-02	Duplicate username is rejected (duplicateUsername)	Sign up a user, then sign up again with the same username and a different e-mail.	400 Bad Request with error "Username already taken".	Pass
BI-03	Login works with username and with e-mail (loginWithUsernameOrEmail)	Sign up a JUDGE, then POST /api/auth/login with the username, then with the e-mail.	Both return 200; role is JUDGE.	Pass
BI-04	Wrong password is rejected (loginWrongPassword)	Sign up a user, then log in with password "nope".	400 with error "Invalid credentials".	Pass
BI-05	/api/auth/me returns the logged-in user (meWithToken)	Sign up an ORGANIZER and call GET /api/auth/me with their token.	200; username matches.	Pass
BI-06	Sign-up without a password is rejected (signupMissingPassword)	POST /api/auth/signup with no password field.	400 Bad Request.	Pass
BI-07	Sign-up with an invalid e-mail is rejected (signupInvalidEmail)	POST /api/auth/signup with e-mail "not-an-email".	Expected 400. Actual: 200, and the account was created (DEF-13).	Fail
BI-08	Newsletter subscribe, repeat and invalid e-mail (newsletter)	POST /api/newsletter/subscribe with fan@test.dms twice, then with "broken@".	First: 200 "Successfully subscribed". Second: 200 "already subscribed". Invalid: 400.	Pass

RbacIntegrationTest: Role-based access control (RBAC)

How this group was set up: Before the tests: sign up one organizer, one judge and two debaters through the API; the organizer creates a tournament with two schools and the judge. Each test then calls the API with one of these users' tokens, or with no token.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-09	Organizer-only: DELETE /api/tournaments/{id} as ORGANIZER	Send DELETE /api/tournaments/{id} with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-10	Organizer-only: DELETE /api/tournaments/{id} as JUDGE	Send DELETE /api/tournaments/{id} with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-11	Organizer-only: DELETE /api/tournaments/{id} as DEBATER	Send DELETE /api/tournaments/{id} with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-12	Organizer-only: DELETE /api/tournaments/{id} as no login	Send DELETE /api/tournaments/{id} with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-13	Organizer-only: POST /api/matches as ORGANIZER	Send POST /api/matches with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-14	Organizer-only: POST /api/matches as JUDGE	Send POST /api/matches with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-15	Organizer-only: POST /api/matches as DEBATER	Send POST /api/matches with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-16	Organizer-only: POST /api/matches as no login	Send POST /api/matches with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-17	Organizer-only: POST /api/score-sheets/{id}/reopen as ORGANIZER	Send POST /api/score-sheets/{id}/reopen with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-18	Organizer-only: POST /api/score-sheets/{id}/reopen as JUDGE	Send POST /api/score-sheets/{id}/reopen with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-19	Organizer-only: POST /api/score-sheets/{id}/reopen as DEBATER	Send POST /api/score-sheets/{id}/reopen with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-20	Organizer-only: POST /api/score-sheets/{id}/reopen as no login	Send POST /api/score-sheets/{id}/reopen with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-21	Organizer-only: POST /api/score-templates as ORGANIZER	Send POST /api/score-templates with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-22	Organizer-only: POST /api/score-templates as JUDGE	Send POST /api/score-templates with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-23	Organizer-only: POST /api/score-templates as DEBATER	Send POST /api/score-templates with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-24	Organizer-only: POST /api/score-templates as no login	Send POST /api/score-templates with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-25	Organizer-only: POST /api/tournaments as ORGANIZER	Send POST /api/tournaments with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-26	Organizer-only: POST /api/tournaments as JUDGE	Send POST /api/tournaments with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-27	Organizer-only: POST /api/tournaments as DEBATER	Send POST /api/tournaments with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-28	Organizer-only: POST /api/tournaments as no login	Send POST /api/tournaments with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-29	Organizer-only: POST /api/tournaments/{id}/generate-next-round as ORGANIZER	Send POST /api/tournaments/{id}/generate-next-round with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-30	Organizer-only: POST /api/tournaments/{id}/generate-next-round as JUDGE	Send POST /api/tournaments/{id}/generate-next-round with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-31	Organizer-only: POST /api/tournaments/{id}/generate-next-round as DEBATER	Send POST /api/tournaments/{id}/generate-next-round with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-32	Organizer-only: POST /api/tournaments/{id}/generate-next-round as no login	Send POST /api/tournaments/{id}/generate-next-round with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-33	Organizer-only: POST /api/tournaments/{id}/judges as ORGANIZER	Send POST /api/tournaments/{id}/judges with the organizer's token.	Must pass the security check: status is not 401/403 and the body is not "Access Denied". (A 400/404 from business rules, e.g. a missing record, is fine here.)	Pass
BI-34	Organizer-only: POST /api/tournaments/{id}/judges as JUDGE	Send POST /api/tournaments/{id}/judges with the judge's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-35	Organizer-only: POST /api/tournaments/{id}/judges as DEBATER	Send POST /api/tournaments/{id}/judges with the debater's token.	Must be blocked (status 400–403) with "Access Denied" from the role check. Observed: 400 "Access Denied" — blocked, although 403 would be the correct code (DEF-10).	Pass
BI-36	Organizer-only: POST /api/tournaments/{id}/judges as no login	Send POST /api/tournaments/{id}/judges with no token.	Must be blocked (status 400–403). Observed: 403 — blocked, although 401 would be the correct code (DEF-05).	Pass
BI-37	Public endpoint: GET /api/tournaments	GET /api/tournaments with no token.	Returns 200 OK without login.	Pass
BI-38	Public endpoint: GET /api/news	GET /api/news with no token.	Returns 200 OK without login.	Pass
BI-39	Public endpoint: GET /api/users/top-debaters	GET /api/users/top-debaters with no token.	Returns 200 OK without login.	Pass
BI-40	Public endpoint: GET /api/users/organizers	GET /api/users/organizers with no token.	Returns 200 OK without login.	Pass
BI-41	Public endpoint: GET /api/search?query=a	GET /api/search?query=a with no token.	Returns 200 OK without login.	Pass
BI-42	Public endpoint: GET /api/matches/live	GET /api/matches/live with no token.	Expected 200. Actual: 403 — the public home page cannot load live matches (DEF-11).	Fail
BI-43	Private endpoint needs login: GET /api/notifications	GET /api/notifications with no token.	Request is blocked (401 or 403).	Pass
BI-44	Private endpoint needs login: GET /api/messages	GET /api/messages with no token.	Request is blocked (401 or 403).	Pass
BI-45	Private endpoint needs login: GET /api/calendar	GET /api/calendar with no token.	Request is blocked (401 or 403).	Pass
BI-46	Private endpoint needs login: GET /api/connections	GET /api/connections with no token.	Request is blocked (401 or 403).	Pass
BI-47	Private endpoint needs login: GET /api/notifications/unread-count	GET /api/notifications/unread-count with no token.	Request is blocked (401 or 403).	Pass
BI-48	Private endpoint needs login: GET /api/matches/1	GET /api/matches/1 with no token.	Request is blocked (401 or 403).	Pass
BI-49	Forged token rejected: "garbage"	GET /api/notifications with "Authorization: Bearer "garbage"".	Request is blocked (401 or 403).	Pass
BI-50	Forged token rejected: a JWT with a made-up signature for organizer1	GET /api/notifications with "Authorization: Bearer a JWT with a made-up signature for organizer1".	Request is blocked (401 or 403).	Pass
BI-51	Private endpoint works with a valid token (privateEndpointWithToken)	Debater calls GET /api/notifications with their token.	200 OK.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-52	Expired token gets 401 (expiredTokenGets401)	Build a token for the debater, correctly signed with the test secret but expired 1 second ago; call GET /api/notifications.	Expected 401 (so the frontend logs the user out). Actual: 403 (DEF-05).	Fail
BI-53	Blocked role gets 403 (deniedRoleGets403)	Debater sends POST /api/tournaments.	Expected 403. Actual: 400 "Access Denied" (DEF-10).	Fail
BI-54	Missing token gets 401 (missingTokenGets401)	POST /api/matches with no token.	Expected 401. Actual: 403 (DEF-05).	Fail
BI-55	/api/auth/me without a token gets 401 (meWithoutToken)	GET /api/auth/me with no token.	Expected 401. Actual: 400 caused by a NullPointerException (DEF-12).	Fail

TournamentWorkflowIntegrationTest: Full workflows

How this group was set up: Each test signs up its own fresh users through the API, so tests do not affect each other.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-56	Complete tournament lifecycle (fullTournamentLifecycle)	1) Sign up an organizer, 2 judges, 2 debaters. 2) Organizer creates a tournament (Royal College vs Trinity College, both judges). 3) GET the tournament. 4) Organizer creates a match with both judges. 5) Judge 1 checks notifications. 6) Judge 1 submits 70 vs 72; match checked; judge 2 submits 65 vs 74; both choose debater B as best speaker. 7) Read match, stats and leaderboard. 8) Judge 1 tries to submit again.	Tournament has 2 schools and judge codes JUDGE-001/002; match is round 1, UPCOMING; judge 1 has "New Judging Assignment"; after only judge 1 the match is still UPCOMING; after both it is COMPLETED, winner Trinity College, best speaker debater B; debater B: 1 win, 1 best-speaker; debater A: 1 loss; judge 1: 1 match judged; leaderboard: Trinity first with 2 points; repeat submission: 400 "already submitted".	Pass
BI-57	Connection request → accept → block (connectionFlow)	Debater A requests B; B checks status; B accepts; A lists connections and count; A blocks B; B tries to send a request to A.	B sees PENDING_RECEIVED; accept returns 200; A's list contains B; count is 1; block returns 200; B's request gets 400 "Action blocked".	Pass
BI-58	Messages, diary, calendar and news (socialContent)	A messages B; B reads messages. A writes a diary post; B likes and comments; anyone reads A's diary; B (a judge) tries to verify it. A creates a calendar event and lists it. A posts a news article; anyone reads news.	B sees "Good luck!"; like and comment return 200; public diary shows "Round 1"; judge verify is refused (400); calendar shows "Practice"; news shows "Finals announced".	Pass
BI-59	Search finds tournaments by name (search)	Organizer creates "Zebra Invitational"; anonymous GET /api/search?query=Zebra.	200; results include "Zebra Invitational".	Pass
BI-60	Score-sheet template can be read back (scoreTemplateReadBack)	Organizer creates a tournament with a template named "Default"; then GET /api/score-templates/{id}.	Expected 200 containing "Default". Actual: 400 "Type definition error ... ByteBuddyInterceptor" (DEF-03).	Fail
BI-61	A submitted score sheet can be read back (submissionReadBack)	Create tournament and match; the judge submits; then GET /api/score-sheets/{match}/{judge}.	Expected 200. Actual: 400 with the same serialization error (DEF-04).	Fail
BI-62	Missing records return 404 (notFound)	GET tournament 999999, match 999999 (logged in) and user 999999.	All three return 404 Not Found.	Pass

OwnershipSecurityIntegrationTest: Ownership and data exposure

How this group was set up: Before the tests: sign up debaters alice and bob, organizers A and B, and a judge. Organizer A creates a tournament containing alice, bob and the judge.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
BI-63	Cannot edit another user's profile (cannotEditOthersProfile)	alice sends PUT /api/users/{bob's id} with bio "hacked by alice".	Expected 403. Actual: 200 and bob's bio was changed (DEF-01).	Fail
BI-64	Can edit own profile (canEditOwnProfile)	alice sends PUT /api/users/{alice's id} with bio "Debater from Kandy".	200; bio is updated.	Pass
BI-65	Cannot post a comment as someone else (cannotImpersonateInDiscussion)	alice sends POST /api/discussion with userId = bob's id.	Expected 403. Actual: 200 and the comment is shown as written by bob (DEF-02).	Fail
BI-66	Cannot delete another user's comment (cannotDeleteOthersComment)	bob posts a comment; alice sends DELETE /api/discussion/{id}.	Expected 403. Actual: 204 and the comment is deleted (DEF-07).	Fail
BI-67	Organizer cannot delete another organizer's tournament (cannotDeleteOtherOrganizersTournament)	Organizer A creates a tournament; organizer B sends DELETE /api/tournaments/{id}.	Expected 403. Actual: 204 and the tournament is deleted (DEF-06).	Fail
BI-68	Cannot mark another user's notification read (cannotReadOthersNotification)	Get alice's first notification id; bob sends PUT /api/notifications/{id}/read.	Expected 403. Actual: 200 (DEF-09).	Fail
BI-69	Judge impersonation gets 403 (judgeImpersonationStatusCode)	A second judge submits a score sheet using the assigned judge's id.	Expected 403. Actual: 400 — the attempt is blocked, but with the wrong status code (DEF-10).	Fail
BI-70	Public user list hides e-mail addresses (publicUserListHidesEmails)	Anonymous GET /api/users.	Expected no e-mail addresses. Actual: every user's e-mail is listed (DEF-08).	Fail

5. Frontend component tests (Vitest + React Testing Library)

Vitest runs each React component inside jsdom, a simulated browser. React Testing Library renders the component and finds elements the way a user would: by placeholder text, button name or visible text. It then types and clicks with fireEvent and checks what appears on screen. Network calls are replaced by Vitest mocks, so no backend is needed. Each test starts with an empty localStorage.

LoginPage.test.tsx: Login page

How this group was set up: LoginPage is rendered in a simulated browser (jsdom) inside an in-memory router. The login API, AuthContext and toast messages are replaced with Vitest mocks so each response can be controlled. Input is typed with fireEvent.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-01	shows the role the user selected on the previous screen	Render with selectedRole = DEBATER.	The text "Signing in as DEBATER" is displayed.	Pass
FE-02	rejects an empty form without calling the API	Click Sign In without typing anything.	Toast "Please fill all required fields"; the login API is never called.	Pass
FE-03	shows the server error message when login fails	Mock the API to fail with "Invalid credentials"; type a user and wrong password; submit.	Toast "Invalid credentials"; the session is not stored.	Pass
FE-04	logs a DEBATER in, stores the session and opens their dashboard	Mock the API to return a token and a user with this role; type username and password; submit.	API called with the typed values; login() stores the token and user; toast "Login successful!"; the page navigates to /dashboard/<role>.	Pass
FE-05	logs a JUDGE in, stores the session and opens their dashboard	Mock the API to return a token and a user with this role; type username and password; submit.	API called with the typed values; login() stores the token and user; toast "Login successful!"; the page navigates to /dashboard/<role>.	Pass
FE-06	logs a ORGANIZER in, stores the session and opens their dashboard	Mock the API to return a token and a user with this role; type username and password; submit.	API called with the typed values; login() stores the token and user; toast "Login successful!"; the page navigates to /dashboard/<role>.	Pass
FE-07	toggles password visibility	Click the eye button next to the password field.	Field type changes from "password" to "text".	Pass

SignupPage.test.tsx: Sign-up and role selection pages

How this group was set up: Pages are rendered in jsdom with an in-memory router; the sign-up API, AuthContext and toasts are mocked. The selected role is JUDGE.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-08	rejects mismatched passwords before calling the API	Fill the form with "password123" and "password124"; submit.	Toast "Passwords do not match"; API not called.	Pass
FE-09	rejects passwords shorter than 6 characters	Fill the form with password "abc" twice; submit.	Toast "Password must be at least 6 characters"; API not called.	Pass
FE-10	shows judge-only fields when the JUDGE role is selected	Render with role JUDGE.	The "Expertise" field is visible.	Pass
FE-11	creates the account with the selected role and opens the dashboard	Mock the API to succeed; fill a valid form; submit.	API called with username, e-mail, role JUDGE and password; login() is called; page navigates to the dashboard.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-12	shows the server error when the username is taken	Mock the API to fail with "Username already taken"; submit.	That message is shown as an error toast.	Pass
FE-13	choosing Debater remembers the role and goes to login	Open /role-select and click the role card.	setSelectedRole is called with the role in capitals; the login page opens.	Pass
FE-14	choosing Judge remembers the role and goes to login	Open /role-select and click the role card.	setSelectedRole is called with the role in capitals; the login page opens.	Pass
FE-15	choosing Organizer remembers the role and goes to login	Open /role-select and click the role card.	setSelectedRole is called with the role in capitals; the login page opens.	Pass

App.test.tsx: Protected routes (frontend access rules)

How this group was set up: The real App with its real route rules and a real AuthProvider is rendered at a given URL. A logged-in user is simulated by putting a token and user in localStorage. Page layouts are replaced by markers so only the access rule is tested. The final URL is read from the router.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-16	logged-out visitor opening /dashboard/organizer is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-17	logged-out visitor opening /dashboard/judge is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-18	logged-out visitor opening /dashboard/debater is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-19	logged-out visitor opening /create-tournament is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-20	logged-out visitor opening /notifications is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-21	logged-out visitor opening /messages is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-22	logged-out visitor opening /score-sheet/1/2 is sent to role selection	Open the protected URL with nothing in localStorage.	The router ends on /role-select.	Pass
FE-23	/dashboard/organizer is refused for a DEBATER and redirects home	Store a user with the listed role and open the URL.	The router ends on "/" (the page is not shown).	Pass
FE-24	/dashboard/organizer is refused for a JUDGE and redirects home	Store a user with the listed role and open the URL.	The router ends on "/" (the page is not shown).	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-25	/create-tournament is refused for a DEBATER and redirects home	Store a user with the listed role and open the URL.	The router ends on "/" (the page is not shown).	Pass
FE-26	/dashboard/judge is refused for a ORGANIZER and redirects home	Store a user with the listed role and open the URL.	The router ends on "/" (the page is not shown).	Pass
FE-27	/dashboard/debater is refused for a JUDGE and redirects home	Store a user with the listed role and open the URL.	The router ends on "/" (the page is not shown).	Pass
FE-28	/dashboard/organizer is allowed for a ORGANIZER	Store a user with the listed role and open the URL.	The URL stays the same and the protected content is rendered.	Pass
FE-29	/create-tournament is allowed for a ORGANIZER	Store a user with the listed role and open the URL.	The URL stays the same and the protected content is rendered.	Pass
FE-30	/dashboard/judge is allowed for a JUDGE	Store a user with the listed role and open the URL.	The URL stays the same and the protected content is rendered.	Pass
FE-31	/dashboard/debater is allowed for a DEBATER	Store a user with the listed role and open the URL.	The URL stays the same and the protected content is rendered.	Pass
FE-32	/notifications is allowed for a DEBATER	Store a user with the listed role and open the URL.	The URL stays the same and the protected content is rendered.	Pass
FE-33	unknown URLs fall back to the home page	Open /this/does/not/exist.	The router ends on "/".	Pass

AuthContext.test.tsx: AuthContext (session state)

How this group was set up: The real AuthProvider is rendered around a small probe component that displays the current state and has buttons for login, logout, choose role and update profile. Other browser tabs are simulated by changing localStorage and firing a "storage" event, which is what the browser does.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-34	starts logged out when nothing is stored	Render with empty localStorage.	isAuthenticated is false.	Pass
FE-35	restores a saved session from localStorage on page reload	Put a token and user in localStorage, then render.	isAuthenticated is true and the username is "debater1".	Pass
FE-36	survives corrupted user data in localStorage	Put invalid JSON in dms_user, then render.	The app renders and treats the user as logged out.	Pass
FE-37	login stores token and user; logout clears everything	Click login, pick role JUDGE, then click logout.	After login the token is stored; the role is stored; after logout the token, user and role are all removed.	Pass
FE-38	updateUser merges changes and persists them	Log in, then update the bio.	The new bio is shown and saved to localStorage.	Pass
FE-39	warns when another tab logs in as a different user (multi-login conflict fix)	Log in as debater1; simulate another tab storing judge1's session.	"User Account Switched" modal appears naming judge1; the current tab now shows judge1.	Pass

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-40	warns when another tab logs out (session expired fix)	Log in; simulate another tab removing the session; click Acknowledge.	"Session Expired / Logged Out" appears; state becomes logged out; the modal closes on Acknowledge.	Pass
FE-41	useAuth outside the provider throws a clear error	Render a component that calls useAuth() without AuthProvider.	Throws "useAuth must be used within AuthProvider".	Pass

axios.test.ts: API client (axios interceptors)

How this group was set up: The real axios instance is used, but its network adapter is replaced so each test decides the HTTP status and can inspect the outgoing request. window.location is replaced so redirects can be observed.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-42	attaches the stored JWT as a Bearer token	Store token JWT-123 and make a request.	The request header is "Authorization: Bearer JWT-123".	Pass
FE-43	sends no Authorization header when logged out	Make a request with empty storage.	No Authorization header is sent.	Pass
FE-44	on 401 clears the session and sends the user to /login	Store a session; the server answers 401.	Token and user are removed from storage; the browser is sent to /login.	Pass
FE-45	on 403 keeps the session (only 401 is treated as an expired login)	Store a session; the server answers 403.	Token stays; no redirect. (This is why DEF-05 matters: the backend sends 403 for expired logins.)	Pass

utils.test.tsx: Utilities and toast

How this group was set up: Pure functions are called directly; the toast is rendered with the real ToastProvider and Vitest fake timers.

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
FE-46	uses an explicit tournament id	Route a notification with tournamentId 7 (with and without a matchId).	Both give /tournament/7.	Pass
FE-47	extracts the tournament id from a match code in the message	Message contains "MATCH-42-3".	Gives /tournament/42.	Pass
FE-48	falls back to the notifications page	"Tournament Created" and "Connection Accepted" notifications without ids.	Both give /notifications.	Pass
FE-49	returns empty string for missing urls	Call with null and undefined.	Returns an empty string.	Pass
FE-50	leaves absolute, blob and data urls untouched	Call with https, http, blob and data URLs.	Each is returned unchanged.	Pass
FE-51	turns a relative backend path into a path on the backend origin	Call with /api/users/1/profile-picture.	Returns the backend path.	Pass
FE-52	shows a message and hides it after 4 seconds	Show "Saved!", then advance the fake clock 4.1 seconds.	The message is visible, then gone.	Pass

6. API end-to-end tests (Postman collection run by Newman)

The backend jar was started on port 8081 against a separate PostgreSQL database called dms_test, so the development data in dms_db was never touched. Newman, the command-line runner for Postman, ran the 69 requests below in order as one story. Values created by one request, such as login tokens, user ids, the tournament id and the match id, are saved in environment variables and reused by later requests. A unique run id is added to every username, so the collection can be re-run at any time. Every request also has one extra check: the response must arrive in under 2000 ms. That check is not repeated in the table.

01 Health & public pages

Endpoints a logged-out visitor uses on the home page.

ID	Request	What was sent	Checks (and failure detail)	Result
API-01	List tournaments (public)	GET /api/tournaments Sent as: No login	- Status is 200 - Body is an array	Pass
API-02	Top debaters (public)	GET /api/users/top-debaters Sent as: No login	Status is 200	Pass
API-03	News feed (public)	GET /api/news Sent as: No login	Status is 200	Pass
API-04	Live matches (used by public home page)	GET /api/matches/live Sent as: No login	Visitors can load live matches (200) Failed: Visitors can load live matches (200) — expected response to have status code 200 but got 403	Fail

02 Authentication

Sign-up for every role, login with username/email, error handling.

ID	Request	What was sent	Checks (and failure detail)	Result
API-05	Sign up organizer (organizer)	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role, bio	- Status is 200 - Returns a JWT token - Role is ORGANIZER - Password hash is never returned	Pass
API-06	Sign up judge (judge1)	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role, bio	- Status is 200 - Returns a JWT token - Role is JUDGE - Password hash is never returned	Pass
API-07	Sign up judge (judge2)	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role, bio	- Status is 200 - Returns a JWT token - Role is JUDGE - Password hash is never returned	Pass

ID	Request	What was sent	Checks (and failure detail)	Result
API-08	Sign up debater (debaterA)	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role, bio	- Status is 200 - Returns a JWT token - Role is DEBATER - Password hash is never returned	Pass
API-09	Sign up debater (debaterB)	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role, bio	- Status is 200 - Returns a JWT token - Role is DEBATER - Password hash is never returned	Pass
API-10	Duplicate username is rejected	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role	- Status is 400 - Error explains the problem	Pass
API-11	Invalid e-mail is rejected at signup	POST /api/auth/signup Sent as: No login Body: JSON with fullName, username, email, password, role	Invalid e-mail gives 400 Failed: Invalid e-mail gives 400 — expected response to have status code 400 but got 200	Fail
API-12	Login with username	POST /api/auth/login Sent as: No login Body: JSON with usernameOrEmail, password	- Status is 200 - Logged in as the judge	Pass
API-13	Login with e-mail	POST /api/auth/login Sent as: No login Body: JSON with usernameOrEmail, password	Status is 200	Pass
API-14	Login with seeded demo account (organizer1)	POST /api/auth/login Sent as: No login Body: usernameOrEmail=organizer1, password=password123	Status is 200	Pass
API-15	Login with wrong password	POST /api/auth/login Sent as: No login Body: JSON with usernameOrEmail, password	- Status is 400 - Generic 'Invalid credentials' message	Pass
API-16	Current user (/auth/me)	GET /api/auth/me Sent as: Debater A	- Status is 200 - Returns the logged-in user	Pass
API-17	Current user without token	GET /api/auth/me Sent as: No login	No token gives 401 Unauthorized Failed: No token gives 401 Unauthorized — expected response to have status code 401 but got 400	Fail

03 Role-based access control

Organizer-only actions attempted by other roles and by anonymous callers.

ID	Request	What was sent	Checks (and failure detail)	Result
API-18	Debater tries to create a tournament	POST /api/tournaments Sent as: Debater A Body: name=Debater Cup, debateType=TRADITIONAL, tournamentType=LEAGUE	- Request is blocked (no 2xx) - Blocked by the role check - Uses HTTP 403 Forbidden Failed: Uses HTTP 403 Forbidden — expected response to have status code 403 but got 400	Fail
API-19	Judge tries to create a match	POST /api/matches Sent as: Judge 1 Body: tournamentId=1, propositionSchoolId=1, oppositionSchoolId=2, topic=x	- Request is blocked (no 2xx) - Uses HTTP 403 Forbidden Failed: Uses HTTP 403 Forbidden — expected response to have status code 403 but got 400	Fail
API-20	Anonymous tries to create a tournament	POST /api/tournaments Sent as: No login Body: name=Anon Cup, debateType=TRADITIONAL, tournamentType=LEAGUE	- Request is blocked (no 2xx) - Uses HTTP 401 Unauthorized Failed: Uses HTTP 401 Unauthorized — expected response to have status code 401 but got 403	Fail
API-21	Forged token is rejected	GET /api/notifications Sent as: Custom token built in the pre-request script	Forged token is blocked	Pass
API-22	Expired token is rejected with 401	GET /api/notifications Sent as: Custom token built in the pre-request script	- Expired token is blocked - Uses HTTP 401 so the frontend logs the user out Failed: Uses HTTP 401 so the frontend logs the user out — expected response to have status code 401 but got 403	Fail

04 Tournament lifecycle

Organizer builds a tournament; judges score; results propagate to stats and leaderboard.

ID	Request	What was sent	Checks (and failure detail)	Result
API-23	Organizer creates tournament with schools, judges and template	POST /api/tournaments Sent as: Organizer Body: JSON with name, debateType, tournamentType, schools, name, debaterIds, name, debaterIds	- Status is 200 - Two schools created - Judges get sequential codes - Tournament starts ACTIVE	Pass
API-24	Tournament is visible publicly	GET /api/tournaments/{{tournamentId}} Sent as: No login	- Status is 200 - Correct name	Pass
API-25	Organizer's tournaments list	GET /api/tournaments/organizer/{{organizerId}} Sent as: No login	- Status is 200 - Contains the new tournament	Pass
API-26	Adding the same judge twice is rejected	POST /api/tournaments/{{tournamentId}}/judges Sent as: Organizer Body: judgeId=judge1Id	- Status is 400 - Explains duplicate	Pass

ID	Request	What was sent	Checks (and failure detail)	Result
API-27	Score template can be read back (judge score sheet)	GET /api/score-templates/{{tournamentId}} Sent as: Judge 1	• Template loads (200) • Template has the organizer's criteria Failed: Template loads (200) — expected response to have status code 200 but got 400; Template has the organizer's criteria — expected '{"timestamp":"2026-09-25T21:19:17.452..." to include 'Matter'	Fail
API-28	A school cannot debate itself	POST /api/matches Sent as: Organizer Body: tournamentId=tournamentId, propositionSchoolId=schoolA, oppositionSchoolId=schoolA, topic=x	• Status is 400	Pass
API-29	Organizer creates a match with two judges	POST /api/matches Sent as: Organizer Body: tournamentId=tournamentId, propositionSchoolId=schoolA, oppositionSchoolId=schoolB, topic=This house would ban homework, judgeIds=[...], startTime=2026-10-01T10:00:00	• Status is 200 • Round 1, UPCOMING • Match code format MATCH-{tournament}-{n}	Pass
API-30	Judge receives assignment notification	GET /api/notifications Sent as: Judge 1	• Status is 200 • Has 'New Judging Assignment'	Pass
API-31	Debater sees the match on the tournament page	GET /api/tournaments/{{tournamentId}}/matches Sent as: No login	• Status is 200 • One match	Pass
API-32	Judge 2 cannot submit on behalf of judge 1	POST /api/score-sheets/submit Sent as: Judge 2 Body: matchId=matchId, judgeId=judge1Id, propositionTotal=99, oppositionTotal=1	• Impersonation is blocked • Uses HTTP 403 Forbidden Failed: Uses HTTP 403 Forbidden — expected response to have status code 403 but got 400	Fail
API-33	Judge 1 submits scores	POST /api/score-sheets/submit Sent as: Judge 1 Body: matchId=matchId, judgeId=judge1Id, propositionScoresJson={}, oppositionScoresJson={}, propositionTotal=70, oppositionTotal=72, selectedBestSpeakerId=debaterBId, comments=Close debate	• Status is 200 • Confirmation message	Pass
API-34	Match stays open until all judges submit	GET /api/matches/{{matchId}} Sent as: Organizer	• Status is 200 • Still UPCOMING	Pass
API-35	Judge 1 can see their submitted sheet	GET /api/score-sheets/{{matchId}}/{{judge1Id}} Sent as: Judge 1	• Submission loads (200) Failed: Submission loads (200) — expected response to have status code 200 but got 400	Fail

ID	Request	What was sent	Checks (and failure detail)	Result
API-36	Judge 2 submits scores	POST /api/score-sheets/submit Sent as: Judge 2 Body: matchId=matchId, judgeId=judge2Id, propositionScoresJson={}, oppositionScoresJson={}, propositionTotal=65, oppositionTotal=74, selectedBestSpeakerId=debaterBId	Status is 200	Pass
API-37	Match is completed with winner and best speaker	GET /api/matches/{matchId} Sent as: Organizer	- Status is 200 - Status COMPLETED - Trinity wins on average score - Best speaker chosen by judges' votes	Pass
API-38	Judge cannot submit twice	POST /api/score-sheets/submit Sent as: Judge 1 Body: matchId=matchId, judgeId=judge1Id, propositionTotal=1, oppositionTotal=1	- Status is 400 - Explains duplicate	Pass
API-39	Leaderboard updated	GET /api/tournaments/{tournamentId}/leaderboard Sent as: No login	- Status is 200 - Trinity first with 2 points	Pass
API-40	Winning debater stats updated	GET /api/stats/debater/{debaterBId} Sent as: No login	- Status is 200 1 win, 1 best-speaker award	Pass
API-41	Losing debater stats updated	GET /api/stats/debater/{debaterAId} Sent as: No login	- Status is 200 1 loss	Pass
API-42	Judge stats updated	GET /api/stats/judge/{judge1Id} Sent as: No login	- Status is 200 1 match judged	Pass
API-43	Next round needs at least two winners	POST /api/tournaments/{tournamentId}/generate-next-round Sent as: Organizer	- Status is 400 - Explains why	Pass

05 Social features

Connections, messages, diaries, calendar, news, notifications, search, newsletter.

ID	Request	What was sent	Checks (and failure detail)	Result
API-44	Debater A sends connection request to B	POST /api/connections/request/{debaterBId} Sent as: Debater A	- Status is 200 - PENDING	Pass
API-45	B sees the pending request	GET /api/connections/pending Sent as: Debater B	- Status is 200 - Has one pending	Pass
API-46	A cannot accept their own request	PUT /api/connections/accept/{connectionId} Sent as: Debater A	Status is 400	Pass
API-47	B accepts	PUT /api/connections/accept/{connectionId} Sent as: Debater B	- Status is 200 - ACCEPTED	Pass

ID	Request	What was sent	Checks (and failure detail)	Result
API-48	Connection status is ACCEPTED	GET /api/connections/status/{{debaterBId}} Sent as: Debater A	• ACCEPTED	Pass
API-49	A sends a message to B	POST /api/messages Sent as: Debater A Body: receiverId=debaterBId, text=Well played today!	• Status is 200	Pass
API-50	B reads messages	GET /api/messages Sent as: Debater B	• Status is 200 • Message delivered	Pass
API-51	A writes a diary post	POST /api/diaries Sent as: Debater A Body: title=Lessons from the final, content=Rebuttals win debates.	• Status is 200	Pass
API-52	B likes the diary post	POST /api/diaries/{{diaryId}}/like Sent as: Debater B	• Status is 200 • Like counted	Pass
API-53	B comments on the diary post	POST /api/diaries/{{diaryId}}/comment Sent as: Debater B Body: comment=Great insight!	• Status is 200	Pass
API-54	Debater cannot verify a diary post (organizer-only)	PUT /api/diaries/{{diaryId}}/verify?verified=true Sent as: Debater B	• Blocked	Pass
API-55	Organizer verifies the diary post	PUT /api/diaries/{{diaryId}}/verify?verified=true Sent as: Organizer	• Status is 200 • Verified	Pass
API-56	Public diary list for A	GET /api/diaries/user/{{debaterAId}} Sent as: No login	• Status is 200 • Post listed	Pass
API-57	Create calendar event	POST /api/calendar Sent as: Judge 1 Body: title=Judging briefing, eventType=JUDGING_ASSIGNMENT, startTime=2026-10-01T09:00:00, endTime=2026-10-01T09:30:00, reminderEnabled=true	• Status is 200	Pass
API-58	Calendar lists the event	GET /api/calendar Sent as: Judge 1	• Event present	Pass
API-59	Post news article	POST /api/news Sent as: Organizer Body: title=Postman Nationals results, category=LATEST_NEWS, content=Trinity College won the opening round.	• Status is 200	Pass
API-60	Unread notification count	GET /api/notifications/unread-count Sent as: Judge 1	• Status is 200 • Count is a number > 0	Pass
API-61	Mark own notification read	PUT /api/notifications/{{judge1NotificationId}}/read Sent as: Judge 1	• Status is 200	Pass

ID	Request	What was sent	Checks (and failure detail)	Result
API-62	Search by tournament name	GET /api/search?query=Postman Sent as: No login	- Status is 200 - Finds the tournament	Pass
API-63	Newsletter subscribe	POST /api/newsletter/subscribe Sent as: No login Body: JSON with email	- Status is 200 - Subscribed	Pass
API-64	Newsletter rejects bad e-mail	POST /api/newsletter/subscribe Sent as: No login Body: email=broken@	Status is 400	Pass

06 Ownership & privacy

Logged-in users must not change or see data that belongs to someone else.

ID	Request	What was sent	Checks (and failure detail)	Result
API-65	Debater A edits debater B's profile	PUT /api/users/{{debaterBId}} Sent as: Debater A Body: bio=Edited by someone else	Cannot edit another user's profile (403) Failed: Cannot edit another user's profile (403) — expected response to have status code 403 but got 200	Fail
API-66	Debater B posts a discussion comment as debater A	POST /api/discussion Sent as: Debater B Body: tournamentId=tournamentId, userId=debaterAId, comment=Posted under someone else's name	Cannot post as another user (403) Failed: Cannot post as another user (403) — expected response to have status code 403 but got 200	Fail
API-67	Judge 2 deletes a comment they don't own	DELETE /api/discussion/{{forgedCommentId}} Sent as: Judge 2	Cannot delete another user's comment (403) Failed: Cannot delete another user's comment (403) — expected response to have status code 403 but got 204	Fail
API-68	Debater marks judge's notification as read	PUT /api/notifications/{{judge1NotificationId}}/read Sent as: Debater A	Cannot modify another user's notification (403) Failed: Cannot modify another user's notification (403) — expected response to have status code 403 but got 200	Fail
API-69	Anonymous user list hides e-mail addresses	GET /api/users Sent as: No login	- Status is 200 - No e-mail addresses exposed to visitors Failed: No e-mail addresses exposed to visitors — expected true to deeply equal false	Fail

7. Browser end-to-end tests (Playwright on Microsoft Edge)

Playwright started the backend (port 8081, database dms_test) and the Vite frontend (port 5174, pointed at that backend), then controlled a real Microsoft Edge browser. Before the run, a global setup step created four fresh users through the API: an organizer, a judge and two debaters, all with unique names. Some tests log in through the real login form. Others create the session directly by writing the token to localStorage, which is what the app itself does after login, to save time. A screenshot is taken at the end of every test, and a full trace is kept for every failure.

01-auth.spec.ts: Authentication and access (browser)

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
E2E-01	organizer logs in through the UI and lands on their dashboard	Open /role-select, click the role card, type username and password on /login, click Sign In.	URL becomes /dashboard/<role> and the page shows the first word of the user's display name.	Pass
E2E-02	judge logs in through the UI and lands on their dashboard	Open /role-select, click the role card, type username and password on /login, click Sign In.	URL becomes /dashboard/<role> and the page shows the first word of the user's display name.	Pass
E2E-03	debaterA logs in through the UI and lands on their dashboard	Open /role-select, click the role card, type username and password on /login, click Sign In.	URL becomes /dashboard/<role> and the page shows the first word of the user's display name.	Pass
E2E-04	wrong password shows an error and stays on the login page	On /login type a real username and a wrong password; click Sign In.	"Invalid credentials" appears and the URL stays /login.	Pass
E2E-05	new user can sign up and is taken to their dashboard	Choose Debater; on /signup fill full name, a unique username, e-mail, password twice; click Create Account.	URL becomes /dashboard/debater.	Pass
E2E-06	logged-out visitor cannot open the organizer dashboard	Logged out, open /dashboard/organizer.	Redirected to /role-select.	Pass
E2E-07	debater cannot open the create-tournament page	Logged in as a debater, open /create-tournament.	Redirected to the home page.	Pass
E2E-08	user can log out from the navbar menu	Log in through the UI; open the avatar menu; click Log Out; confirm in the dialog.	The login token is removed from the browser.	Pass

02-session.spec.ts: Multi-tab sessions (browser)

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
E2E-09	logging in as another user in a second tab warns the first tab	Tab A: logged in as a debater on the dashboard. Tab B (same browser profile): log in as the judge.	Tab A shows "User Account Switched" with the judge's username.	Pass
E2E-10	logging out in a second tab tells the first tab the session ended	Tab A on the dashboard; tab B removes the stored session.	Tab A shows "Session Expired / Logged Out".	Pass
E2E-11	an expired token sends the user back to login	Replace the stored token with an expired/invalid one and open /notifications.	Expected redirect to /login. Actual: the user stays on /notifications because the server answers 403, not 401 (DEF-05).	Fail

03-tournament.spec.ts: Tournament workflow (browser, run in order)

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
E2E-12	organizer creates a tournament with the 6-step wizard	As the organizer open /create-tournament. Step 1: type a name, choose Asian Parliamentary. Step 2: keep Knockout. Step 3: name "Royal College", search and add debater A; add a school "Trinity College" with debater B. Step 4: search and add the judge. Step 5: rename the first criterion to "Persuasion". Step 6: check the review and click Create Tournament.	Review shows "2 schools, 2 debaters" and "1 judges"; after creating, the tournament page opens with the new name.	Pass
E2E-13	a visitor can find the new tournament with search	Open /search?q=<tournament name>.	The tournament name is shown.	Pass
E2E-14	judge opens the score sheet for an assigned match	Create a match through the API with the judge; as the judge open /score-sheet/<match>/<judge>.	The match topic and the Submit Score Sheet button are shown.	Pass
E2E-15	judge submits the score sheet	Fill every score box (30 for proposition, 25 for opposition); click Submit Score Sheet.	A "submitted" confirmation is shown.	Pass
E2E-16	match result appears on the tournament page	Read the match through the API; open the tournament page.	Match status is COMPLETED; the tournament page shows Royal College.	Pass

04-public.spec.ts: Public pages (browser)

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
E2E-17	/ loads without errors	Open the page while recording JavaScript errors.	Expected text is shown and no uncaught JavaScript error occurred.	Pass
E2E-18	/news loads without errors	Open the page while recording JavaScript errors.	Expected text is shown and no uncaught JavaScript error occurred.	Pass
E2E-19	/about loads without errors	Open the page while recording JavaScript errors.	Expected text is shown and no uncaught JavaScript error occurred.	Pass
E2E-20	/scoring loads without errors	Open the page while recording JavaScript errors.	Expected text is shown and no uncaught JavaScript error occurred.	Pass
E2E-21	search finds seeded debaters	Open /search?q=debater1.	debater1 appears in the results.	Pass
E2E-22	home page loads live matches for visitors without API errors	Open / logged out while recording every API response; wait for the network to go quiet.	Expected no failed API calls. Actual: GET /api/matches/live returned 403 (DEF-11).	Fail
E2E-23	layout works on a phone-sized screen	Set the window to 375 × 812 and open /.	The page is not wider than the screen (no horizontal scrolling).	Pass

05-score-sheet.spec.ts: Score sheet (browser, independent tests)

ID	Test	How it was conducted	What is checked (and actual outcome if failed)	Result
E2E-24	score sheet shows the organizer's custom criteria	Through the API the organizer creates a tournament with criteria "Persuasion" and "Evidence" and a match; the judge opens the score sheet.	Expected column "Persuasion" and no "Rebuttal". Actual: the built-in columns Matter/Manner/Method/Rebuttal/Teamwork are shown (DEF-03; see screenshot in the report).	Fail
E2E-25	after submitting, reopening the sheet shows "already submitted"	The judge submits through the API, then opens the score sheet page.	Expected "Score Sheet Submitted". Actual: an empty form is shown again (DEF-04).	Fail

8. Lighthouse audits

The frontend was built for production (vite build) and served with vite preview. Google Lighthouse then loaded each page in headless Microsoft Edge using the desktop preset and scored four categories from 0 to 100. These are audits rather than pass/fail tests. Problems they found are recorded as DEF-11 and DEF-15 in the testing report.

ID	Page	Scores	Speed measurements
LH-1	/	Performance 99 · Accessibility 94 · Best practices 96 · SEO 83	First paint 0.7 s · Largest paint 0.9 s · Blocking 0 ms · Layout shift 0.000
LH-2	/login	Performance 100 · Accessibility 81 · Best practices 100 · SEO 83	First paint 0.7 s · Largest paint 0.7 s · Blocking 0 ms · Layout shift 0.001
LH-3	/news	Performance 99 · Accessibility 94 · Best practices 100 · SEO 83	First paint 0.7 s · Largest paint 0.9 s · Blocking 0 ms · Layout shift 0.000
LH-4	/role-select	Performance 99 · Accessibility 94 · Best practices 100 · SEO 83	First paint 0.6 s · Largest paint 0.8 s · Blocking 0 ms · Layout shift 0.000